SEC2_APM1110-SA1_Part1_GROUP_6-DABU, L; MONTILLA, A

GitHub: https://github.com/dabulira20/Prob-ProbDi-FAs/tree/main/SA1

2026-10-11

Contents

Overview **1**

Item 1: Probability that a product is defective **2**

Validation and calculation functions 2

Interactive prompt version 3

Demonstration 3

Item 2: Front end for discrete random variables **4**

Helper functions 4

Demonstration: validity checks 6

Demonstration: univariate distribution 7

Demonstration: bivariate distribution 8

The interactive Shiny front end 11

Item 3: Search simulation **14**

Simulated pdf, mean and variance 14

Conditional distribution after three unsuccessful searches 15

Memoryless (Markov) property 16

References **17**

Overview

This document contains the complete solution to Section I (Items 1 to 3). Each item includes the code, example output, and a brief interpretation.

Item	Topic
1	Probability that a randomly selected product is defective
2	Front end for univariate and bivariate discrete random variables

Item	Topic
3	Simulation of searches until a key phrase is found

Requirements: R >= 4.0. The Shiny app in Item 2 also needs `install.packages("shiny")`.

Item 1: Probability that a product is defective

Method. Factory i produces a proportion x_i of the product ($\sum x_i = 1$, $0.10 \le x_i \le 0.40$) with defective rate y_i ($\sum y_i = 0.12$, $0.01 \le y_i \le 0.05$). By the law of total probability

$$P(D) = \sum_{i=1}^{3} P(D \mid F_i) P(F_i) = \sum_{i=1}^{3} x_i y_i, \qquad P(F_i \mid D) = \frac{x_i y_i}{P(D)}.$$

Validation and calculation functions

```r
# Parse "0.2, 0.3 0.5" or "20%, 30%, 50%" into proportions
parse_probs <- function(txt) {
  parts <- strsplit(trimws(txt), "[,;[:space:]]+")[[1]]
  parts <- parts[parts != ""]
  is_pct <- grepl("%", parts, fixed = TRUE)
  vals <- suppressWarnings(as.numeric(sub("%", "", parts, fixed = TRUE)))
  vals[is_pct] <- vals[is_pct] / 100
  vals
}

# Returns "" if valid, otherwise a message explaining the problem
validate_vector <- function(v, lo, hi, total, n = 3) {
  if (length(v) != n || any(is.na(v))) return(sprintf("Enter exactly %d
   ↪  numeric values.", n))
  if (any(v < lo - TOL | v > hi + TOL)) return(sprintf("Every value must lie
   ↪  in [%g, %g].", lo, hi))
  if (abs(sum(v) - total) > TOL) return(sprintf("Values sum to %.6f but
   ↪  must sum to %g.", sum(v), total))
  ""
}

# Core calculation
defect_probability <- function(x, y) {
  joint <- x * y
  p_def <- sum(joint)
  list(p_defective = p_def, joint = joint, posterior = joint / p_def)
}
```

Interactive prompt version

This function asks the user for the inputs and repeats the prompt until they are valid. It is not run when knitting. Call `run_item1_interactive()` in the console.

```
ask_vector <- function(label, symbol, lo, hi, total) {
  repeat {
    cat(sprintf("\nEnter the 3 %s values (%s1, %s2, %s3), each in [%g, %g],
     ↪  summing to %g:\n",
                label, symbol, symbol, symbol, lo, hi, total))
    v <- parse_probs(readline("> "))
    msg <- validate_vector(v, lo, hi, total)
    if (msg == "") return(v)
    cat("  [Invalid]", msg, "\n")
  }
}

run_item1_interactive <- function() {
  x <- ask_vector("production proportion", "x", 0.10, 0.40, 1)
  y <- ask_vector("defective rate", "y", 0.01, 0.05, 0.12)
  res <- defect_probability(x, y)
  print(data.frame(Factory = paste("Factory", 1:3), x = x, y = y,
                   `x*y` = res$joint, `P(F|D)` = round(res$posterior, 4),
                    ↪  check.names = FALSE))
  cat(sprintf("\nP(defective) = %.6f\n", res$p_defective))
}
run_item1_interactive()
```

Demonstration

Validation of invalid inputs:

```
tests <- list(
  "x out of range (0.50, 0.30, 0.20)" = list(parse_probs("0.50, 0.30,
   ↪  0.20"), 0.10, 0.40, 1),
  "x does not sum to 1 (0.2, 0.3, 0.4)" = list(parse_probs("0.2, 0.3,
   ↪  0.4"), 0.10, 0.40, 1),
  "y out of range (0.06, 0.03, 0.03)"   = list(parse_probs("0.06, 0.03,
   ↪  0.03"), 0.01, 0.05, 0.12),
  "y does not sum to 0.12 (1%, 2%, 3%)"  = list(parse_probs("1%, 2%, 3%"),
   ↪  0.01, 0.05, 0.12),
  "valid x (25%, 35%, 40%)"             = list(parse_probs("25%, 35%,
   ↪  40%"), 0.10, 0.40, 1)
)
for (nm in names(tests)) {
  t <- tests[[nm]]
  msg <- validate_vector(t[[1]], t[[2]], t[[3]], t[[4]])
  cat(sprintf("%-40s -> %s\n", nm, ifelse(msg == "", "VALID", msg)))
}
```

```
## x out of range (0.50, 0.30, 0.20)      -> Every value must lie in [0.1, 0.4].
## x does not sum to 1 (0.2, 0.3, 0.4)    -> Values sum to 0.900000 but must sum to 1.
```

```
## y out of range (0.06, 0.03, 0.03)      -> Every value must lie in [0.01, 0.05].
## y does not sum to 0.12 (1%, 2%, 3%)    -> Values sum to 0.060000 but must sum to 0.12.
## valid x (25%, 35%, 40%)                     -> VALID
```

Example calculation with valid inputs $x = (0.25, 0.35, 0.40)$ and $y = (0.03, 0.04, 0.05)$:

```
x <- c(0.25, 0.35, 0.40)
y <- c(0.03, 0.04, 0.05)
stopifnot(validate_vector(x, 0.10, 0.40, 1) == "", validate_vector(y, 0.01,
 ↪  0.05, 0.12) == "")
res <- defect_probability(x, y)

knitr::kable(data.frame(
  Factory = paste("Factory", 1:3),
  `Proportion x` = x, `Defect rate y` = y,
  `x * y` = res$joint,
  `P(Factory | Defect)` = round(res$posterior, 4),
  check.names = FALSE
))
```

Factory	Proportion x	Defect rate y	x * y	P(Factory | Defect)
Factory 1	0.25	0.03	0.0075	0.1807
Factory 2	0.35	0.04	0.0140	0.3373
Factory 3	0.40	0.05	0.0200	0.4819

```
cat(sprintf("P(defective) = %.4f (%.2f%%)\n", res$p_defective, 100 *
 ↪  res$p_defective))
```

```
## P(defective) = 0.0415 (4.15%)
```

Interpretation. The overall defect rate is a weighted average of the factory defect rates, with production shares as weights. Factory 3 makes 40% of the output but accounts for about 48% of the defective items, because it has both the largest share and the highest defect rate.

Item 2: Front end for discrete random variables

Helper functions

All calculations and validity checks used by the app are defined here so they can also be demonstrated in this document.

```
parse_nums <- function(txt) {
  parts <- strsplit(trimws(txt), "[,;[:space:]]+")[[1]]
  parts <- parts[parts != ""]
  v <- suppressWarnings(as.numeric(parts))
  if (length(parts) == 0 || any(is.na(v))) stop("Please enter numeric
   ↪  values only (separated by commas or spaces).")
```

```r
  v
}

check_probs <- function(p, label = "Probabilities") {
  if (any(p < 0 | p > 1)) stop(sprintf("%s must all lie in [0, 1].", label))
  if (abs(sum(p) - 1) > 1e-6) stop(sprintf("%s must sum to 1 (they
   ↪  currently sum to %.6f).", label, sum(p)))
  invisible(TRUE)
}

make_pmf <- function(x, p, label = "Probabilities") {
  if (length(x) != length(p)) stop(sprintf("Number of values (%d) and
   ↪  probabilities (%d) differ.", length(x), length(p)))
  if (anyDuplicated(x)) stop("Values must be distinct.")
  check_probs(p, label)
  o <- order(x)
  list(x = x[o], p = p[o])
}

pmf_stats <- function(d) {
  m <- sum(d$x * d$p)
  v <- sum((d$x - m)^2 * d$p)
  list(mean = m, var = v, sd = sqrt(v))
}

plot_pmf <- function(d, main) {
  plot(d$x, d$p, type = "h", lwd = 4, col = "steelblue", ylim = c(0,
   ↪  max(d$p) * 1.1),
       xlab = "x", ylab = "P(X = x)", main = main)
  points(d$x, d$p, pch = 16, col = "steelblue")
}

plot_cdf <- function(d, main) {
  plot(stepfun(d$x, c(0, cumsum(d$p))), verticals = FALSE, pch = 16, col =
   ↪  "firebrick",
       xlim = range(d$x) + c(-1, 1), ylim = c(0, 1), xlab = "x", ylab =
        ↪  "F(x)", main = main)
}

# Joint table: one row per line (rows = X values), columns = Y values
parse_joint <- function(txt, nx, ny) {
  lines <- strsplit(trimws(txt), "\n")[[1]]
  lines <- lines[nzchar(trimws(lines))]
  rows <- lapply(lines, parse_nums)
  if (length(rows) != nx) stop(sprintf("Joint table needs %d rows; found
   ↪  %d.", nx, length(rows)))
  if (any(vapply(rows, length, 1L) != ny)) stop(sprintf("Every row needs %d
   ↪  entries.", ny))
  do.call(rbind, rows)
}

# Build the full bivariate object: joint, marginals
make_bivariate <- function(x, y, J) {
  if (anyDuplicated(x) || anyDuplicated(y)) stop("X values must be distinct,
   ↪  and Y values must be distinct.")
```

```r
  check_probs(as.vector(J), "Joint probabilities")
  ox <- order(x); oy <- order(y)
  x <- x[ox]; y <- y[oy]; J <- J[ox, oy, drop = FALSE]
  dimnames(J) <- list(paste0("x=", x), paste0("y=", y))
  list(x = x, y = y, J = J, mx = list(x = x, p = rowSums(J)), my = list(x =
    ↪  y, p = colSums(J)))
}

# Conditional distribution of X | Y = v  (type "XgY")  or  Y | X = v  (type
 ↪  "YgX")
conditional <- function(b, type, v) {
  if (type == "XgY") {
    j <- match(v, b$y); if (is.na(j)) stop("Conditioning value not found
 ↪  among Y values.")
    if (b$my$p[j] == 0) stop("P(Y = y) = 0, conditional distribution
     ↪  undefined.")
    list(dist = list(x = b$x, p = b$J[, j] / b$my$p[j]), lab = sprintf("X |
     ↪  Y = %g", v))
  } else {
    i <- match(v, b$x); if (is.na(i)) stop("Conditioning value not found
 ↪  among X values.")
    if (b$mx$p[i] == 0) stop("P(X = x) = 0, conditional distribution
     ↪  undefined.")
    list(dist = list(x = b$y, p = b$J[i, ] / b$mx$p[i]), lab = sprintf("Y |
     ↪  X = %g", v))
  }
}
```

Demonstration: validity checks

```r
try_it <- function(expr) tryCatch({ force(expr); "VALID" }, error =
 ↪  function(e) conditionMessage(e))

cat("Probability 1.2 in list      :", try_it(make_pmf(1:3, c(1.2, -0.1,
 ↪  -0.1))), "\n")
```

```
## Probability 1.2 in list      : Probabilities must all lie in [0, 1].
```

```r
cat("Probabilities sum to 0.9     :", try_it(make_pmf(1:3, c(0.3, 0.3,
 ↪  0.3))), "\n")
```

```
## Probabilities sum to 0.9    : Probabilities must sum to 1 (they currently sum to 0.90000
```

```r
cat("Length mismatch              :", try_it(make_pmf(1:3, c(0.5, 0.5))),
 ↪  "\n")
```

```
## Length mismatch          : Number of values (3) and probabilities (2) differ.
```

```
cat("Non-numeric input            :", try_it(parse_nums("1, two, 3")), "\n")
```

```
## Non-numeric input         : Please enter numeric values only (separated by commas or spa
```

```
cat("Valid input                  :", try_it(make_pmf(1:4, c(0.1, 0.2, 0.3,
 ↪  0.4))), "\n")
```

```
## Valid input                   : VALID
```

Demonstration: univariate distribution

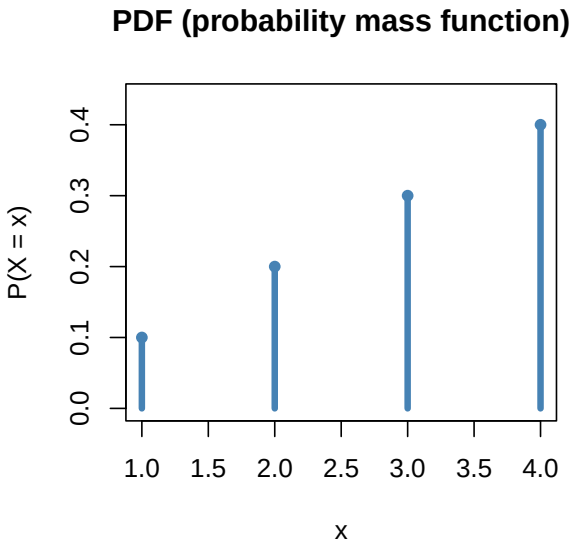
```
d <- make_pmf(parse_nums("1, 2, 3, 4"), parse_nums("0.1, 0.2, 0.3, 0.4"))
s <- pmf_stats(d)
knitr::kable(data.frame(x = d$x, `P(X=x)` = d$p, `F(x)` = cumsum(d$p),
 ↪  check.names = FALSE))
```

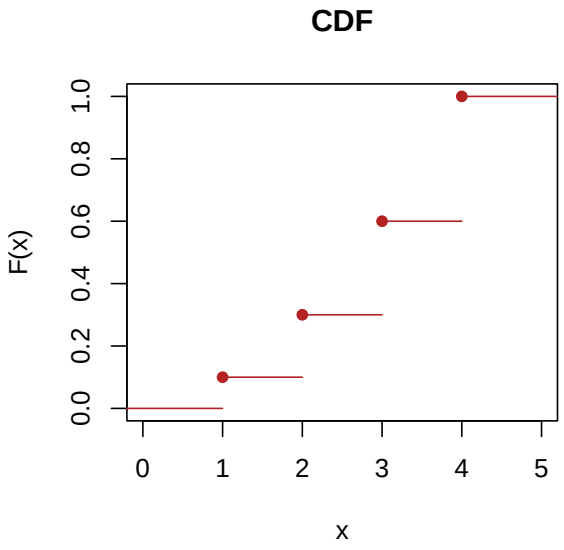
x	P(X=x)	F(x)
1	0.1	0.1
2	0.2	0.3
3	0.3	0.6
4	0.4	1.0

```
cat(sprintf("Mean = %.4f, Variance = %.4f, SD = %.4f\n", s$mean, s$var,
 ↪  s$sd))
```

```
## Mean = 3.0000, Variance = 1.0000, SD = 1.0000
```

```
par(mfrow = c(1, 2))
plot_pmf(d, "PDF (probability mass function)")
plot_cdf(d, "CDF")
```






```r
par(mfrow = c(1, 1))
```

Demonstration: bivariate distribution

```r
x <- parse_nums("0, 1, 2")
y <- parse_nums("0, 1")
J <- parse_joint("0.10, 0.20\n0.15, 0.25\n0.20, 0.10", length(x), length(y))
b <- make_bivariate(x, y, J)

cat("Joint distribution:\n"); print(b$J)
```

```
## Joint distribution:

##       y=0  y=1
## x=0 0.10 0.20
## x=1 0.15 0.25
## x=2 0.20 0.10
```

```r
cat("\nMarginal of X:\n"); print(data.frame(x = b$mx$x, p = b$mx$p))
```

```
##
## Marginal of X:

##     x   p
## x=0 0 0.3
## x=1 1 0.4
## x=2 2 0.3
```

```r
cat("\nMarginal of Y:\n"); print(data.frame(y = b$my$x, p = b$my$p))
```

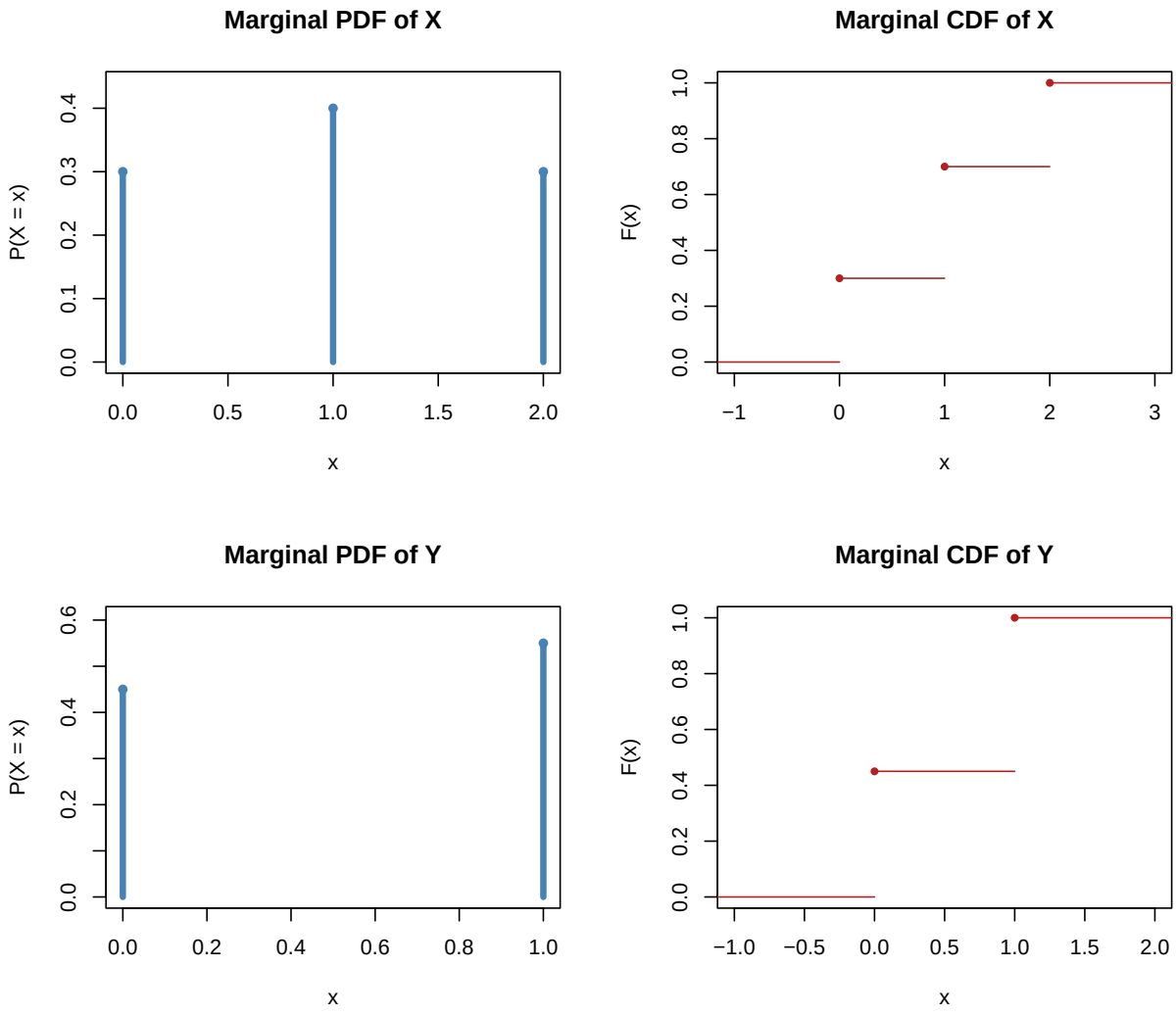
```
##
## Marginal of Y:
```

```
##      y    p
## y=0  0 0.45
## y=1  1 0.55
```

```r
sx <- pmf_stats(b$mx); sy <- pmf_stats(b$my)
cat(sprintf("\nX: mean = %.4f, variance = %.4f\nY: mean = %.4f, variance =
 ↪  %.4f\n",
            sx$mean, sx$var, sy$mean, sy$var))
```

```
##
## X: mean = 1.0000, variance = 0.6000
## Y: mean = 0.5500, variance = 0.2475
```

```r
par(mfrow = c(2, 2))
plot_pmf(b$mx, "Marginal PDF of X"); plot_cdf(b$mx, "Marginal CDF of X")
plot_pmf(b$my, "Marginal PDF of Y"); plot_cdf(b$my, "Marginal CDF of Y")
```




```
par(mfrow = c(1, 1))
```

Conditional distribution of X given $Y = 1$:

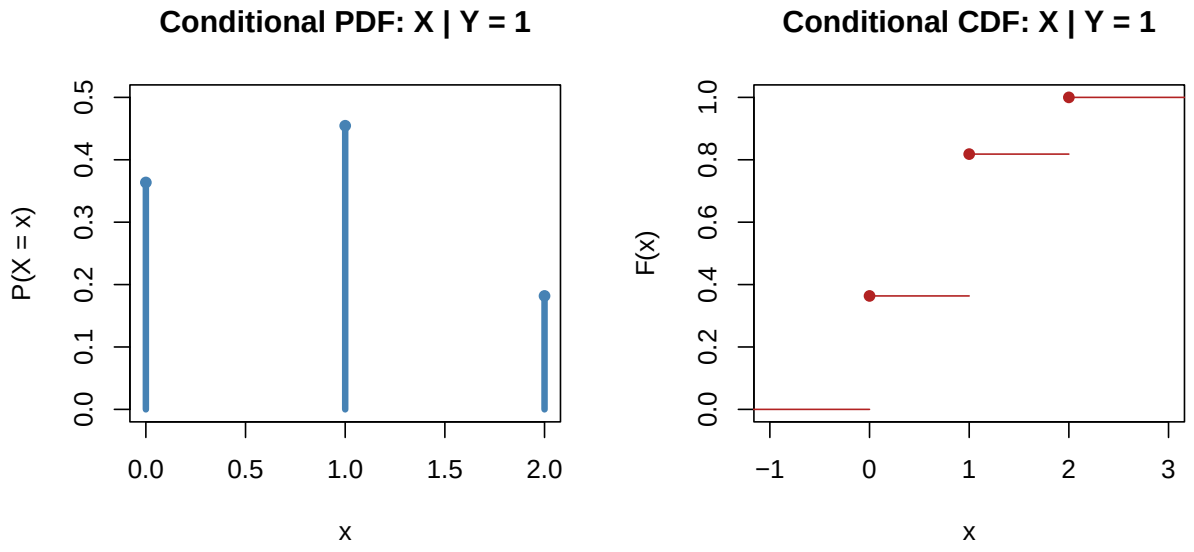
```
cd <- conditional(b, "XgY", 1)
sc <- pmf_stats(cd$dist)
print(data.frame(x = cd$dist$x, `P(X=x|Y=1)` = cd$dist$p, check.names =
  ↪  FALSE))
```

```
##      x P(X=x|Y=1)
## x=0  0  0.3636364
## x=1  1  0.4545455
## x=2  2  0.1818182
```

```
cat(sprintf("%s: mean = %.4f, variance = %.4f\n", cd$lab, sc$mean, sc$var))
```

```
## X | Y = 1: mean = 0.8182, variance = 0.5124
```

```r
par(mfrow = c(1, 2))
plot_pmf(cd$dist, paste("Conditional PDF:", cd$lab))
plot_cdf(cd$dist, paste("Conditional CDF:", cd$lab))
```




```r
par(mfrow = c(1, 1))
```

The interactive Shiny front end

The code below builds the app using the helper functions above. It is not run when knitting. To launch it, run the chunk interactively in RStudio (after running the helper chunk), or copy the code to `app.R` together with the helper functions and call `shiny::runApp()`.

```r
library(shiny)

ui <- fluidPage(
  titlePanel("Discrete Random Variable Calculator"),
  tabsetPanel(
    tabPanel("Univariate",
      sidebarLayout(
        sidebarPanel(
          textInput("u_x", "Values of X (comma separated)", "1, 2, 3, 4"),
          textInput("u_p", "Probabilities (comma separated)", "0.1, 0.2,
            ↪ 0.3, 0.4"),
          helpText("Each probability must be in [0, 1] and they must sum to
            ↪ 1.")
        ),
        mainPanel(
          verbatimTextOutput("u_err"), tableOutput("u_table"),
            ↪ verbatimTextOutput("u_stats"),
          fluidRow(column(6, plotOutput("u_pdf")), column(6,
            ↪ plotOutput("u_cdf")))
```

```
        )
      )),
    tabPanel("Bivariate",
      sidebarLayout(
        sidebarPanel(
          textInput("b_x", "Values of X (rows)", "0, 1, 2"),
          textInput("b_y", "Values of Y (columns)", "0, 1"),
          textAreaInput("b_joint", "Joint probabilities (one row per X
            ↪ value)",
                        "0.10, 0.20\n0.15, 0.25\n0.20, 0.10", rows = 6),
          uiOutput("cond_controls")
        ),
        mainPanel(
          verbatimTextOutput("b_err"),
          h4("Joint distribution"), tableOutput("b_joint_tbl"),
          h4("Marginal distributions"),
          fluidRow(column(6, tableOutput("b_mx")), column(6,
            ↪ tableOutput("b_my"))),
          verbatimTextOutput("b_stats"),
          fluidRow(column(6, plotOutput("bx_pdf")), column(6,
            ↪ plotOutput("bx_cdf"))),
          fluidRow(column(6, plotOutput("by_pdf")), column(6,
            ↪ plotOutput("by_cdf"))),
          h4("Conditional distribution"), tableOutput("b_cond_tbl"),
            ↪ verbatimTextOutput("b_cond_stats"),
          fluidRow(column(6, plotOutput("c_pdf")), column(6,
            ↪ plotOutput("c_cdf")))
        )
      ))
  )
)

server <- function(input, output, session) {
  bad <- function(e) structure(list(msg = conditionMessage(e)), class =
 ↪  "bad")

  # ---- univariate ----
  u_dist <- reactive(tryCatch(make_pmf(parse_nums(input$u_x),
 ↪  parse_nums(input$u_p)), error = bad))
  u_ok <- function() !inherits(u_dist(), "bad")
  output$u_err <- renderText({ d <- u_dist(); if (inherits(d, "bad"))
 ↪  paste("Input error:", d$msg) else "" })
  output$u_table <- renderTable({ req(u_ok()); d <- u_dist()
    data.frame(x = d$x, `P(X=x)` = d$p, `F(x)` = cumsum(d$p), check.names =
      ↪ FALSE) })
  output$u_stats <- renderText({ req(u_ok()); s <- pmf_stats(u_dist())
    sprintf("Mean E[X] = %.4f\nVariance Var(X) = %.4f\nStd. deviation =
      ↪ %.4f", s$mean, s$var, s$sd) })
  output$u_pdf <- renderPlot({ req(u_ok()); plot_pmf(u_dist(), "PDF") })
  output$u_cdf <- renderPlot({ req(u_ok()); plot_cdf(u_dist(), "CDF") })

  # ---- bivariate ----
  b_data <- reactive(tryCatch({
    x <- parse_nums(input$b_x); y <- parse_nums(input$b_y)
```

```
    make_bivariate(x, y, parse_joint(input$b_joint, length(x), length(y)))
  }, error = bad))
  b_ok <- function() !inherits(b_data(), "bad")
  output$b_err <- renderText({ d <- b_data(); if (inherits(d, "bad"))
↪   paste("Input error:", d$msg) else "" })

  output$cond_controls <- renderUI({
    req(b_ok())
    tagList(
      radioButtons("cond_type", "Conditional distribution of:",
                   c("X given Y = y" = "XgY", "Y given X = x" = "YgX")),
      selectInput("cond_val", "Conditioning value", choices = b_data()$y))
  })
  observeEvent(input$cond_type, {
    req(b_ok()); d <- b_data()
    updateSelectInput(session, "cond_val", choices = if (input$cond_type ==
      ↪ "XgY") d$y else d$x)
  })

  output$b_joint_tbl <- renderTable({ req(b_ok()); b_data()$J }, rownames =
↪   TRUE, digits = 4)
  output$b_mx <- renderTable({ req(b_ok()); d <- b_data()$mx; data.frame(x =
↪   d$x, `P(X=x)` = d$p, check.names = FALSE) })
  output$b_my <- renderTable({ req(b_ok()); d <- b_data()$my; data.frame(y =
↪   d$x, `P(Y=y)` = d$p, check.names = FALSE) })
  output$b_stats <- renderText({ req(b_ok()); d <- b_data(); sx <-
↪   pmf_stats(d$mx); sy <- pmf_stats(d$my)
    sprintf("X: mean = %.4f, variance = %.4f\nY: mean = %.4f, variance =
      ↪ %.4f", sx$mean, sx$var, sy$mean, sy$var) })
  output$bx_pdf <- renderPlot({ req(b_ok()); plot_pmf(b_data()$mx,
↪   "Marginal PDF of X") })
  output$bx_cdf <- renderPlot({ req(b_ok()); plot_cdf(b_data()$mx,
↪   "Marginal CDF of X") })
  output$by_pdf <- renderPlot({ req(b_ok()); plot_pmf(b_data()$my,
↪   "Marginal PDF of Y") })
  output$by_cdf <- renderPlot({ req(b_ok()); plot_cdf(b_data()$my,
↪   "Marginal CDF of Y") })

  cond_dist <- reactive({
    req(b_ok(), input$cond_val, input$cond_type)
    res <- tryCatch(conditional(b_data(), input$cond_type,
↪   as.numeric(input$cond_val)), error = bad)
    validate(need(!inherits(res, "bad"), if (inherits(res, "bad")) res$msg
      ↪ else ""))
    res
  })
  output$b_cond_tbl <- renderTable({ cd <- cond_dist()
    data.frame(value = cd$dist$x, probability = cd$dist$p, cumulative =
      ↪ cumsum(cd$dist$p)) })
  output$b_cond_stats <- renderText({ cd <- cond_dist(); s <-
↪   pmf_stats(cd$dist)
    sprintf("%s: mean = %.4f, variance = %.4f", cd$lab, s$mean, s$var) })
  output$c_pdf <- renderPlot({ cd <- cond_dist(); plot_pmf(cd$dist,
↪   paste("Conditional PDF:", cd$lab)) })
```

```
  output$c_cdf <- renderPlot({ cd <- cond_dist(); plot_cdf(cd$dist,
  ↪  paste("Conditional CDF:", cd$lab)) })
}

shinyApp(ui, server)
```

Interpretation. For the univariate example, $E[X] = 3$ and $Var(X) = 1$, and the CDF rises in steps of size equal to each probability until it reaches 1. In the bivariate example the marginals are obtained by summing the joint table over the other variable, and the conditional distribution is a column (or row) of the joint table rescaled by its marginal probability so that it sums to 1. If X and Y were independent, every conditional distribution of X would equal its marginal; comparing the output above shows whether that holds here.

Item 3: Search simulation

Each site contains the key phrase with probability p (set in the YAML `params`, default 0.6). Let X be the number of searches up to and including the first success, so $X \sim$ Geometric(p) with $P(X = k) = (1-p)^{k-1}p$, $E[X] = 1/p$ and $Var(X) = (1-p)/p^2$. The simulation uses 10,000 runs.

```
p <- params$p
N <- params$N
stopifnot(is.numeric(p), p > 0, p <= 1)           # validity check on p

one_search <- function(p) {
  k <- 1
  while (runif(1) >= p) k <- k + 1                 # this site lacks the
   ↪  phrase: try the next
  k
}
X <- replicate(N, one_search(p))
```

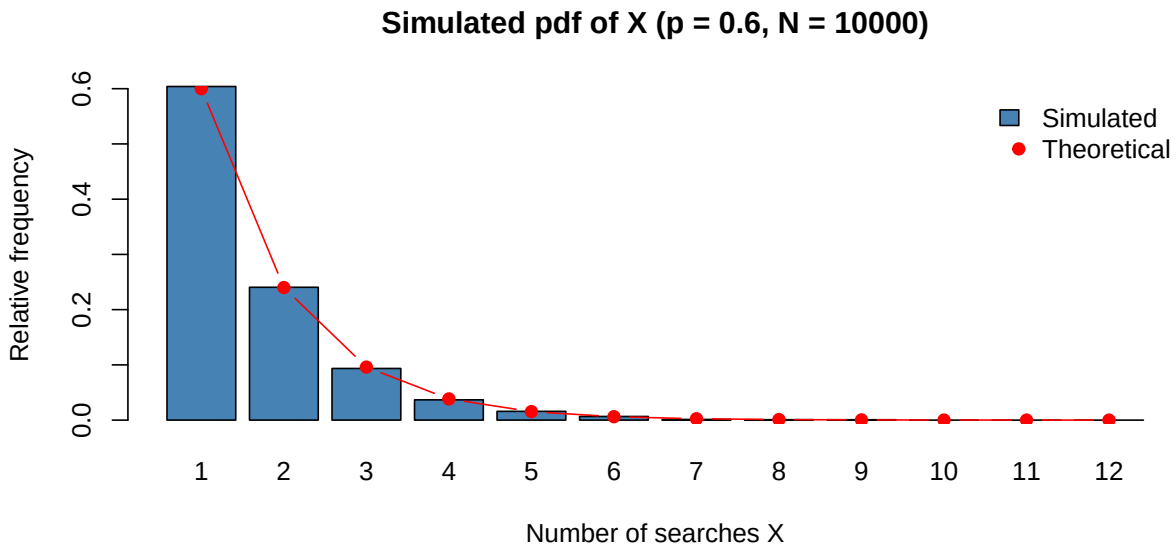
Simulated pdf, mean and variance

```
pdf_sim <- table(factor(X, levels = 1:max(X))) / N
m_sim <- mean(X)
v_sim <- mean((X - m_sim)^2)

knitr::kable(data.frame(
  Statistic = c("Mean", "Variance"),
  Simulated = round(c(m_sim, v_sim), 4),
  Theory    = round(c(1 / p, (1 - p) / p^2), 4)
))
```

Statistic	Simulated	Theory
Mean	1.6538	1.6667
Variance	1.0783	1.1111

```r
k_show <- 1:min(max(X), 12)
mids <- barplot(pdf_sim[k_show], col = "steelblue", xlab = "Number of
 ↪  searches X",
                ylab = "Relative frequency",
                main = sprintf("Simulated pdf of X (p = %g, N = %d)", p, N))
lines(mids, dgeom(k_show - 1, p), type = "b", pch = 19, col = "red")
legend("topright", c("Simulated", "Theoretical"), fill = c("steelblue",
 ↪  NA),
       border = c("black", NA), pch = c(NA, 19), col = c(NA, "red"), bty =
        ↪  "n")
```



Conditional distribution after three unsuccessful searches

```r
n0 <- 3
Xc <- X[X > n0]
Yc <- Xc - n0                                    # further searches still
 ↪  needed
m_c <- mean(Yc)
v_c <- mean((Yc - m_c)^2)

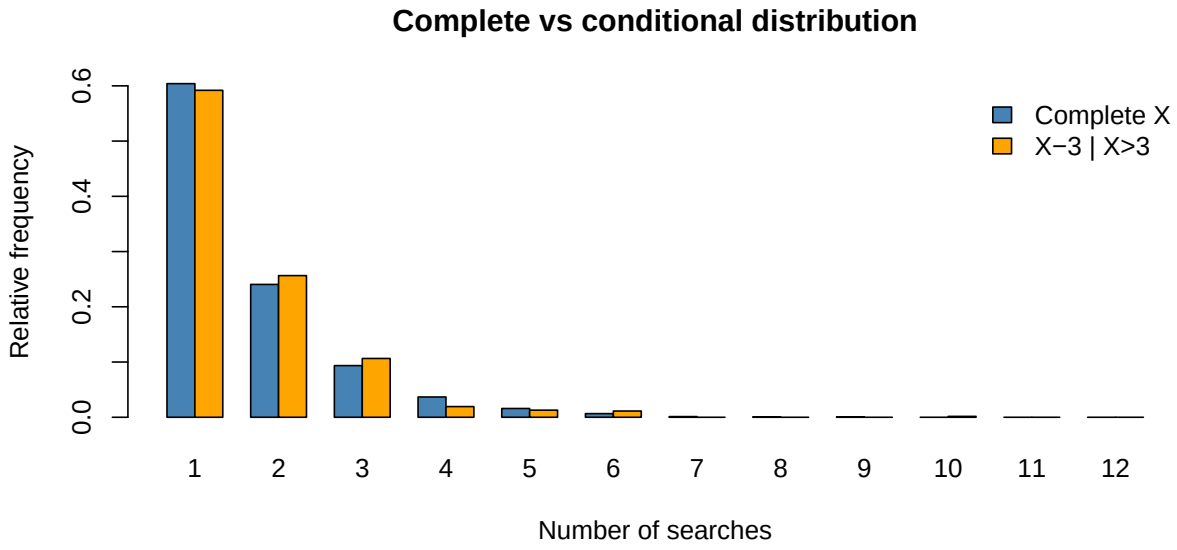
cat(sprintf("Runs with X > %d: %d of %d\n", n0, length(Xc), N))
```

```
## Runs with X > 3: 620 of 10000
```

```r
knitr::kable(data.frame(
  Distribution = c("Complete set (X)", "Conditional (X - 3 | X > 3)"),
  Mean = round(c(m_sim, m_c), 4),
  Variance = round(c(v_sim, v_c), 4)
))
```

Distribution	Mean	Variance
Complete set (X)	1.6538	1.0783
Conditional (X - 3 | X > 3)	1.6500	1.0533

```
pdf_c <- table(factor(Yc, levels = 1:max(X))) / length(Yc)
cmp <- rbind(`Complete X` = as.numeric(pdf_sim[k_show]), `X-3 | X>3` =
 ↪  as.numeric(pdf_c[k_show]))
colnames(cmp) <- k_show
barplot(cmp, beside = TRUE, col = c("steelblue", "orange"), xlab = "Number
 ↪  of searches",
        ylab = "Relative frequency", main = "Complete vs conditional
         ↪  distribution",
        legend.text = TRUE, args.legend = list(x = "topright", bty = "n"))
```



Memoryless (Markov) property

```
res <- data.frame(
  Quantity  = c("(a) P(X=4 | X>3)", "    P(X=1)", "(b) P(X=5 | X>3)", "
   ↪  P(X=2)"),
  Simulated = round(c(mean(X[X > 3] == 4), mean(X == 1), mean(X[X > 3] ==
   ↪  5), mean(X == 2)), 4),
  Theory    = round(c(dgeom(0, p), dgeom(0, p), dgeom(1, p), dgeom(1, p)),
   ↪  4)
)
knitr::kable(res)
```

Quantity	Simulated	Theory
(a) P(X=4 | X>3)	0.5919	0.60
P(X=1)	0.6040	0.60
(b) P(X=5 | X>3)	0.2565	0.24
P(X=2)	0.2405	0.24

Interpretation.

- The simulated pdf decays geometrically and matches the theoretical Geometric(p) mass function. The simulated mean and variance agree with $1/p$ and $(1-p)/p^2$ up to sampling error.
- After three unsuccessful searches, the number of *remaining* searches has essentially the same distribution, mean and variance as the original X, so the conditional and complete distributions are equivalent.
- The estimates satisfy $P(X = 4 \mid X > 3) \approx P(X = 1)$ and $P(X = 5 \mid X > 3) \approx P(X = 2)$. This is the memoryless property $P(X = n + k \mid X > n) = P(X = k)$: earlier failures do not change the chance of success on the next site, because the sites are independent.
- Conditional estimates are noisier because only about $(1-p)^3 N$ runs (640 expected) satisfy $X > 3$.

References

- Ross, S. M. *A First Course in Probability*. Pearson.
- R Core Team. *R: A Language and Environment for Statistical Computing*.
- Chang, W. et al. *shiny: Web Application Framework for R*.